AppPassport.
Know what your AI uses.

I built AppPassport to check an exact AI tool or context packet against attributable, current review evidence. Private memory stays local. Another application can read the evidence directly from Arkiv.

Live application | Source and reproduction | Built by Ajnas NB

10 artifacts	**10/10 protocol calls**	**44/44 tests**
31 chain records	**13/13 chain outcomes**	**Clean SDK install passed**

The public demonstration

1. Open Context handoff and verify the current Qarinah release packet.
2. Change its source snapshot or modify the packet: admission is denied.
3. Select a withdrawn or expired review: admission is denied.
4. Choose an AI tool, check its pinned artifact, then upload changed bytes.
5. Download the SDK and reproduce the decision through the raw RPC.

Why Arkiv is part of the design

The hash identifies bytes. A separate author supplies the current review and can withdraw a particular earlier record. Arkiv provides queryable creator metadata, typed artifact indexes, readonly payloads and expiring storage. A consumer chooses accepted authors and reads their records through its own provider, without depending on the AppPassport API.

A conventional database can implement the same application checks. Arkiv matters when applications share attributable review status through a common external data layer. It is not necessary for local hashing, local memory or every single-company deployment.

What these results establish

The compiler produced genuine Qarinah packets from a synthetic repository. Ten tools initialized, listed their MCP tools and completed one controlled call. The clean consumer installed the SDK tarball with published Qarinah 0.7.0, checked an actual source file, admitted the context and passed the independent CLI check.

These are project-run checks, not an independent security audit. No malware-detection accuracy, upstream endorsement, customer adoption or second LLM inference run is claimed. Both demo wallets are project controlled. Chain metadata and query completeness rely on the RPC provider.

Ten real tool artifacts.

Tool / release	MCP tools	Result	SHA-256 prefix
Qarinah 0.6.0+1aa7022	3	Passed	3c8b3df0d1020a5b
MCP Filesystem 1.0.0	14	Passed	ca60ed89a0acdf91
MCP Memory 1.0.0	9	Passed	22547eddb9544b54
MCP Sequential Thinking 1.0.0	1	Passed	e39382cabd78716a
MCP Everything 1.0.0	14	Passed	05087332283bd470
Playwright MCP 0.0.83	25	Passed	f0014a13361b1ee2
Chrome DevTools MCP 1.10.1	30	Passed	012cbcf6e832d4f6
MCP Git 2026.10.10	12	Passed	5bd3dc5736d67ab0
MCP Fetch 2026.10.10	1	Passed	d722c48bdc1de3e2
MCP Time 2026.10.10	2	Passed	32983d5193af2193

Nine artifacts passed registry integrity checks. Qarinah is a disclosed source snapshot from baseline commit 1aa70228299de902d92fc97214766c0aea02be9d. Full digests, source URLs and the dependency lock are in the public source and JSON reports.

Direct chain evidence

Tiramisu, chain ID 7738577. Namespace: app-passport-v1.
Read-back block: 522895. Evaluated at: 2026-10-13T12:45:41.974Z.
Producer: 0x4FEade3FF513A7954e9f7420aaa7BDEdc0BA4BDC.
Reviewer: 0xd153C0cC313884DA0065615BB411Dfb8b873fE54.

The 31 records include ten tool passports, three context passports, reviews and real withdrawals. Earlier failed browser-client checks are retained in the transaction journal; those negative reviews were withdrawn before fresh reviews were issued. Original Qarinah folders and commits remain unchanged.

Reproduce and inspect

From passport-demo: npm ci --ignore-scripts; npm run build; npm test; npm run fixtures; npm run arkiv:verify; npm run handoff:consume. The independent reader needs no signing wallet. Publication uses free testnet GLM. The public application requires no wallet, payment or login.

Protocol results | SDK and API tests | Chain read-back | Clean consumer | Entity and transaction journal